

Book Finder Robot

Phase 1: Z Calibration Protocol Design

Project	Automated library book-finding robot
Phase	1 of 3 — Vertical (Z) calibration
Devices	Laptop (RTX 4060) + Raspberry Pi + ESP32
Links	WiFi (Laptop ↔ Pi), Classic Bluetooth SPP (Pi ↔ ESP32)
Goal	Find the optimal stepper Z position for each shelf row

This document specifies the communication protocol, message catalog, session flow, and integration plan for Phase 1 of the book-finder robot. It is the single source of truth that both the laptop and the Raspberry Pi implementations must follow.

1. Overview

Phase 1 calibrates the vertical (Z) axis of the robot. The robot's camera mast slowly elevates from the bottom of the bookshelf to the top while the camera captures photos at known stepper positions. The laptop runs AprilTag detection on each photo and computes the exact stepper Z value that corresponds to each row of the shelf.

The result is a small dictionary, saved to disk on the Pi:

```
{
  "0": 519,      // tag 0 (bottom row)  -> stepper Z = 519 steps
  "1": 2840,     // tag 1 (middle row)   -> stepper Z = 2840 steps
  "2": 5160      // tag 2 (top row)      -> stepper Z = 5160 steps
}
```

Phases 2 and 3 (book search scan and navigation, defined later) will load this file and use it to position the camera at any row instantly and reliably, without needing real-time tag detection during normal operation.

Why calibrate once instead of detecting in real time?

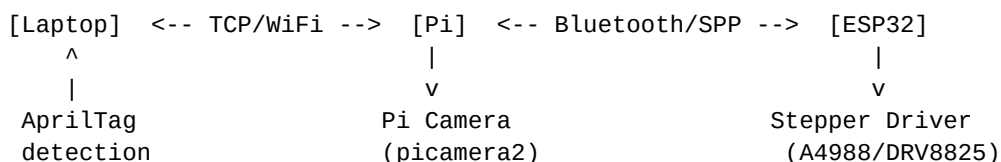
- **Speed:** navigating to row 2 becomes 'go to Z=2840', no vision required.
- **Robustness:** real-time tag detection can fail on motion blur, partial occlusion, or bad lighting. Calibrated values don't have that problem.
- **Precision:** averaging many overlapping detections cancels noise. A single real-time detection might be $\pm 5\text{mm}$ off; the average of 8 is closer to $\pm 0.5\text{mm}$.

2. System Architecture

Three devices, two links, clear separation of responsibility:

Device	Role	Responsibilities
Laptop	Brain (vision + math)	TCP server. Runs AprilTag detection and pose estimation. Computes the optimal
Raspberry Pi	Body (sensors + dispatch)	TCP client to laptop. Captures camera frames. Talks to ESP32 over Bluetooth. E
ESP32	Muscle (motor control)	Bluetooth SPP peripheral. Receives ELEVATE commands and pulses the stepp

Data flow:



Why this split?

- **Laptop is fastest at AI.** RTX 4060 runs AprilTag pose estimation in milliseconds. The Pi 3 would take seconds.
- **Pi is best at real-time I/O.** It owns the camera and the Bluetooth radio. It coordinates timing.
- **ESP32 is best at deterministic hardware control.** Tight stepper pulse timing with no OS jitter.

3. Protocol Design

3.1 Message format

Every message in the system uses the same format: a JSON object on a single line, terminated by a newline character. If the JSON has a **len** field with a positive integer value, exactly that many bytes of binary payload follow immediately after the newline.

Examples:

```
// Plain control message:
{"cmd": "CONTINUE", "step_size": 100}\n

// Message with binary payload (a JPEG photo):
{"type": "CALIB_PHOTO", "seq": 3, "z_steps": 300, "len": 50432}\n
<50432 raw JPEG bytes>
```

3.2 Why this design

Choice	Rationale
JSON for control	Readable, debuggable with any terminal. Adding new fields later does not break existing code.
Newline as delimiter	Easy to resync after a dropped byte: just skip to the next newline.
Length-prefix for binary	JPEG bytes can contain anything (including newlines). The "len" field tells the receiver exactly how many bytes to expect.
Both formats on one TCP socket	Same connection carries commands and photos. Receiver decides which based on the "len" field.
Synchronous flow	After each photo, the Pi waits for the laptop's response before doing anything else. No races.
Detection on the laptop	PaddleOCR / AprilTag on RTX 4060 $\approx$ 140 ms. On Pi 3 $\approx$ 1000+ ms. The 30 ms WiFi transfer time is negligible.

3.3 What is on the wire

The protocol uses two physical transports and three logical channels:

Link	Transport	Logical purpose
Laptop $\leftrightarrow$ Pi	TCP over WiFi, port 5000	High-level calibration commands and photo upload
Pi $\leftrightarrow$ ESP32	Bluetooth Classic SPP (rfcomm)	Low-level motor commands (ELEVATE, STOP, PING)

4. Message Catalog

Every message Phase 1 uses, in full. Field types: int = integer, str = string, dict = object. All fields are required unless noted.

4.1 Laptop → Pi

START_CALIB — begin a calibration session

```
{"cmd": "START_CALIB", "expected_rows": 3, "step_size": 100}
```

Field	Type	Meaning
cmd	str	Always "START_CALIB"
expected_rows	int	Number of shelf rows to find (e.g. 3)
step_size	int	Stepper steps between consecutive captures

CONTINUE — elevate and capture the next photo

```
{"cmd": "CONTINUE", "step_size": 100}
```

Field	Type	Meaning
cmd	str	Always "CONTINUE"
step_size	int	Stepper steps to elevate before the next capture

CALIB_DONE — calibration successful

```
{"cmd": "CALIB_DONE", "z_positions": {"0": 519, "1": 2840, "2": 5160}}
```

Field	Type	Meaning
cmd	str	Always "CALIB_DONE"
z_positions	dict	Map from tag_id → stepper Z (in steps)

CALIB_ERROR — calibration aborted by laptop

```
{"cmd": "CALIB_ERROR", "reason": "missing_1_rows",  
  "partial_z_positions": {"0": 519, "1": 2840}}
```

Field	Type	Meaning
cmd	str	Always "CALIB_ERROR"
reason	str	Human-readable cause
partial_z_positions	dict	Optional. Whatever rows we did find.

4.2 Pi → Laptop

CALIB_PHOTO — calibration photo + metadata

```
{"type": "CALIB_PHOTO", "seq": 7, "z_steps": 700, "len": 48512}\n<48512 raw JPEG bytes>
```

Field	Type	Meaning
type	str	Always "CALIB_PHOTO"
seq	int	Sequence number (0, 1, 2, ...) for logging
z_steps	int	Stepper position WHEN this photo was captured
len	int	Number of bytes of JPEG payload that follow

CALIB_MAX_Z — Pi hit the hardware Z limit

```
{"type": "CALIB_MAX_Z", "z_steps": 10000}
```

Field	Type	Meaning
type	str	Always "CALIB_MAX_Z"
z_steps	int	Final stepper position reached

CALIB_ERROR — Pi-side failure

```
{"type": "CALIB_ERROR", "reason": "esp32_elevate_failed"}
```

Field	Type	Meaning
type	str	Always "CALIB_ERROR"
reason	str	e.g. "esp32_elevate_failed", "camera_failed"

4.3 Pi ↔ ESP32 (Bluetooth)

Pi → ESP32:

```
{"cmd": "PING"}
{"cmd": "ELEVATE", "steps": 100}
{"cmd": "STOP"}
```

ESP32 → Pi:

```
{"type": "PONG"}
{"type": "DONE", "steps_done": 100}
{"type": "ERROR", "reason": "too_many_steps"}
```

Important: ELEVATE is synchronous on the ESP32 side. The ESP32 does NOT acknowledge the command and respond later — it only sends DONE after the stepper has finished moving. This means the Pi knows exactly when the mast is stable enough to capture a non-blurred photo.

5. Calibration Flow

Full sequence of messages and actions during a successful calibration of 3 rows.

Laptop	Pi	ESP32
-----	--	-----
(starts pc_server.py, listens on :5000)		
	(starts pi_client.py)	
	<connect to laptop:5000>	
(accept)		
START_CALIB	■■■■■■■■■■	
	capture photo @ z=0	
	■■■■■■■■■■ CALIB_PHOTO(seq=0, z=0)	
detect tags: none yet		
CONTINUE	■■■■■■■■■■	
	ELEVATE(100) ■■■■	(steps)
	■■■■■ DONE	
	capture photo @ z=100	
	■■■■■■■■■■ CALIB_PHOTO(seq=1, z=100)	
detect tags: none yet		
CONTINUE	■■■■■■■■■■	
	ELEVATE(100) ■■■■	
	■■■■■ DONE	
	capture @ z=200	
	■■■■■■■■■■ CALIB_PHOTO(seq=2, z=200)	
detect tags: [0] ← first tag visible		
CONTINUE	■■■■■■■■■■	
...		
(several captures while tag 0 visible)		
detect tags: [0] z_estimate=519		
detect tags: [0] z_estimate=520		
detect tags: [] ← tag 0 just left frame!		
■■ row 0 DONE, avg z = 519		
CONTINUE	■■■■■■■■■■	
...		
(tag 1 appears, gets sampled, leaves)		
■■ row 1 DONE, avg z = 2840		
...		
(tag 2 appears, gets sampled, leaves)		
■■ row 2 DONE, avg z = 5160		
CALIB_DONE(z_positions={0:519,		
1:2840, 2:5160})	■■■■■■■■■■	
	save z_positions.json	
	disconnect, exit	

5.1 "Row done" detection logic

A row is considered DONE the moment its tag *stops being visible* after having been visible. This is the cleanest signal that the camera has fully passed by the row. The averaged Z estimate at that point is the final answer for that row.

Additionally we require at least **3 detections** of a tag before trusting the average, to filter out single-frame false positives.

5.2 Pose-to-stepper math (laptop side)

For each photo where a tag is detected, the AprilTag library returns the tag's 3D position relative to the camera, in METERS, using the physical tag size and camera intrinsics.

The Y component (vertical offset) tells us how much higher or lower the tag is than the camera's optical axis. To get the stepper Z that would center this tag in the frame:

```
y_offset_cm = tag.pose_y * 100      # meters -> cm
z_optimal   = z_at_capture - round(y_offset_cm * STEPS_PER_CM)

# Sign convention: stepper Z increases <==> mast moves UP
# If tag is ABOVE optical axis (pose_y negative), we need to GO UP,
# i.e. INCREASE z. The minus sign in the formula handles this:
#   y_offset_cm < 0 => -y_offset_cm > 0 => z_optimal > z_at_capture.
```

We collect many `z_optimal` estimates per tag (one per photo where it's visible) and AVERAGE them at the end. This cancels random per-photo noise (motion blur, JPEG quantization, sub-pixel detection jitter).

6. Edge Cases & Failure Modes

Scenario	Detection	Behavior
Mast reaches hardware top before all rows see MAX_Z_STEP	Pi: call row_z_see MAX_Z_STEP	PS sends CALIB_MAX_Z. Laptop sends CALIB_ERROR with what
Tag visible in one photo only (laptop detects)	Laptop: detections < MIN_DETECTIONS_PER_TAG	Estimate is kept but excluded from final
Tag never seen at all	Laptop: tag_id missing from final	Reported in CALIB_ERROR reason field. User should reposition
Bluetooth drops mid-calibration	Pi: esp32.elevate() returns False	PS sends CALIB_ERROR with reason "esp32_elevate_failed". Ca
WiFi drops mid-calibration	Either side: stream.recv_message() returns None	Side that notices logs the disconnect. No partial save (we did not
Camera capture fails	Pi: capture_jpeg() raises	Currently uncaught. TODO: wrap in try/except and send CALIB_I
Stepper stalls / loses steps	NOT detected by current protocol	TODO: add encoder feedback or limit switches in a future iteration

7. File Structure

```
phase1_z_calibration/  
  docs/  
    PROTOCOL_DESIGN.pdf      (this document)  
  pc_laptop/  
    protocol.py               (runs on the laptop)  
    pc_server.py              (shared message framing)  
    apriltag_processor.py      (main: orchestrates calibration)  
    camera_calibration.py      (vision: tags -> Z estimates)  
    (ONE-TIME tool, run later)  
  raspberry_pi/  
    protocol.py               (runs on the Pi)  
    pi_client.py              (IDENTICAL copy of laptop's)  
    camera_capture.py          (main: capture loop)  
    esp32_bt.py               (picamera2 wrapper)  
    (Bluetooth link to ESP32)  
  esp32/  
    elevation_controller/  
      elevation_controller.ino  (Arduino sketch)
```

8. Setup & Running

8.1 Install dependencies

Laptop:

```
pip install opencv-python numpy pupil-apriltags
```

Raspberry Pi:

```
# picamera2 is preinstalled on recent Pi OS
pip install pyserial Pillow opencv-python numpy
```

ESP32:

Open `elevation_controller.ino` in the Arduino IDE. Install these libraries via Tools → Manage Libraries:

- ArduinoJson (by Benoit Blanchon, v6.x or v7.x)
- BluetoothSerial (comes with the ESP32 board package)

Select board: Tools → Board → ESP32 Arduino → ESP32 Dev Module. Then edit `STEP_PIN`, `DIR_PIN`, `ENA_PIN` at the top of the sketch to match your wiring, and upload.

8.2 Pair the Pi with the ESP32 (one-time)

```
# On the Pi:
sudo bluetoothctl
> scan on
> (wait for 'BookFinderESP32' to appear, note its MAC address)
> pair <MAC>
> trust <MAC>
> exit

# Then bind a serial port to the ESP32:
sudo rfcomm bind 0 <MAC> 1

# Now /dev/rfcomm0 is the Bluetooth link.
# (You can put the rfcomm bind in /etc/rc.local to auto-bind on boot.)
```

8.3 Network

- Start the laptop's WiFi hotspot.
- Connect the Pi to that hotspot.
- On the laptop, find its IP on that interface (Windows: *ipconfig*; macOS/Linux: *ip addr*). It will typically be 192.168.137.1 on Windows hotspots.
- Test: from the Pi, run *ping <laptop_ip>* and confirm it replies.

8.4 Running a calibration session

Step 1. Position the robot at the right side of the shelf, facing the AprilTags, with the camera at the lowest position.

Step 2. On the laptop:

```
cd pc_laptop/  
python pc_server.py  
# Output: Waiting for Pi to connect on port 5000...
```

Step 3. On the Pi:

```
cd raspberry_pi/  
python pi_client.py <laptop_ip>  
# Example: python pi_client.py 192.168.137.1
```

Step 4. Watch the laptop's terminal. Each photo prints a one-line status with z position, file size, and detected tag IDs. When the session ends successfully, the final Z positions are saved to `z_positions.json` on the Pi.

9. Testing Without Hardware

You can validate the entire PC ↔ Pi protocol flow on a laptop alone, without any actual Pi, camera, or ESP32. This is the fastest way to catch protocol bugs before hardware integration.

9.1 Mock the camera

Create a folder `mock_photos/` next to `pi_client.py` and drop 10-20 JPEG photos in it (ideally photos of your printed AprilTags at different mast heights, captured manually). Then:

```
MOCK_CAMERA=1 python pi_client.py 127.0.0.1
```

9.2 Mock the ESP32

If you don't have the ESP32 paired yet, but you have a real Pi camera:

```
MOCK_ESP32=1 python pi_client.py <laptop_ip>
```

9.3 Mock both (laptop-only end-to-end test)

```
# Terminal 1 (server):  
cd pc_laptop/  
python pc_server.py  
  
# Terminal 2 (client) - same laptop:  
cd raspberry_pi/  
MOCK_CAMERA=1 MOCK_ESP32=1 python pi_client.py 127.0.0.1
```

This runs the full protocol loop using disk photos and pretend elevation. If it works here, the protocol code is correct - any remaining issues are hardware-side.

10. Camera Calibration (Optional, for Later)

Phase 1 works **right now** using approximate camera intrinsics derived from the OV5647 datasheet. Accuracy is roughly ± 2 cm on tag positions, which is fine for identifying which shelf row we're at (rows are ≥ 20 cm apart).

For Phase 2 (book OCR distance precision) you'll want real calibrated intrinsics. The procedure is:

Step 1 — Print a checkerboard.

- Search: *OpenCV 9x6 checkerboard PDF*.
- "9x6" means 9 by 6 INTERNAL CORNERS (i.e. a 10x7 grid of squares).
- Tape FLAT to a stiff backing. Any warp ruins the calibration.
- Measure one square with a ruler and update SQUARE_SIZE_MM in *camera_calibration.py*.

Step 2 — Take ~25 photos with the Pi camera.

- Same resolution you use in production (1280×720 by default).
- Vary the checkerboard's angle, distance, and position in the frame.
- Make sure the entire checkerboard is visible and in focus in each photo.
- Save them as JPEGs to a folder named *calib_photos/*.

Step 3 — Run the calibration.

```
cd pc_laptop/  
python camera_calibration.py
```

- Reprojection error under 1.0 px = good.
- Above 2.0 px = something is wrong (warped board, blurry photos, wrong CHECKERBOARD setting).

Step 4 — Use the result.

The script saves *camera_calib.npz* in the current folder. As long as that file is next to *apriltag_processor.py*, the next *pc_server.py* run will auto-detect and use it (no code changes). The console will print 'Camera intrinsics: calibrated (loaded from camera_calib.npz)' on startup.

11. What's Next (Not in Phase 1)

Phase	Description	New protocol additions needed
2	Book search scan: drive along X, capture X for OCR, take results. PHOTO with X position metadata, SCAN_RESULTS	X for OCR, X for results
3	Navigate to a specific book: user picks a book, robot moves to X, US heartbeat, arrival confirmation.	GO TO, Obstacle, STAY, US
Auto-connect	Bluetooth and WiFi reconnect automatically, system power on, Pi ready, local IP ready	system power on, Pi ready, local IP ready

Phase 1 is intentionally small and complete. Don't add Phase 2/3 messages to the protocol until Phase 1 runs end-to-end on real hardware. Bugs found early are cheaper than bugs found late.